

Sample Design Document — Backstage

This is a teaching sample, and it is imperfect on purpose. It continues the Backstage project from the sample proposal: same theater, same invented evidence. It follows the design document template, and parts of it are genuinely good. Other parts would stall a builder, hide an assumption, or bury the decisions under detail. Read it the way a reviewer would, and find them.

Printable copy: — print it if you'd rather mark up paper.

Four questions to read with: 1. Could you **start building** from this? What would you have to ask first? 2. Where is the **unstated assumption**? 3. Which significant decision has **no alternative considered** — and what would the alternative be? 4. Where is it **over-specified** — what could be cut with no loss?

Backstage — Design Document

Authors	A. Reyes, J. Okafor
Reviewers	M. Tanaka (design-review partner)
Status	Draft
Last updated	Week 6
Links	Proposal (CP3) · repository · issue tracker

Revision history

DATE	AUTHOR	WHAT CHANGED
Week 6, Mon	A. Reyes	Skeleton from the in-class session
Week 6, Wed	J. Okafor	Diagrams from the workshop; data model; tech choices

1. Context and scope

Backstage is a phone-first web catalog for a community theater's costume stock. Wardrobe volunteers search what the theater owns from the shop floor, see a photo and bin location, and check items out to actors against a show. The full argument, interviews, and spike are in the proposal. This document covers the MVP: one theater's costume stock, seeded with 60 real items and tested during a production strike.

Each theater gets its own deployment on a small VPS.

2. Goals and non-goals

Goals

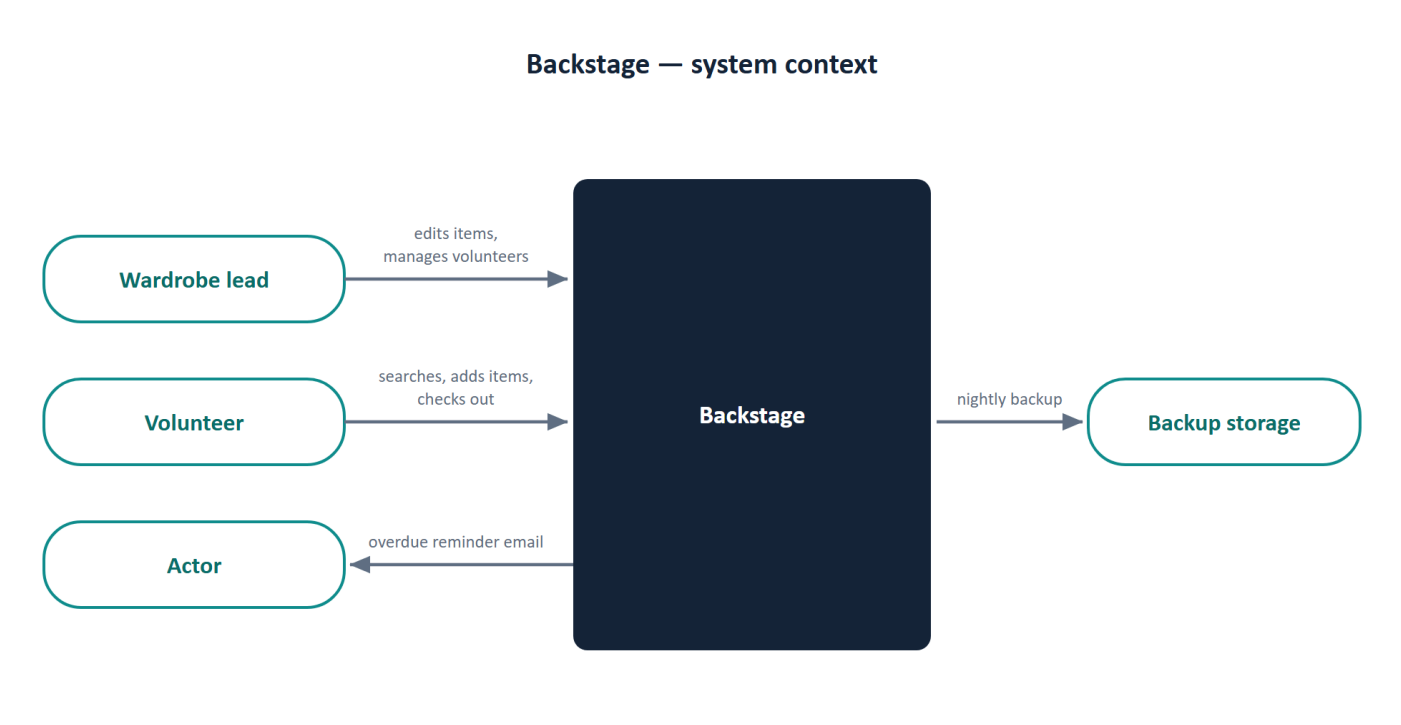
- A volunteer answers “do we own this?” from a phone in under a minute.
- Every checked-out item shows who has it, for which show, and when it's due back.
- A non-technical volunteer can add an item with no verbal instruction.
- The theater can keep running the system after the semester, with no developer on call.

Non-goals

- No ticketing, box office, or budgeting.
- No rental marketplace between theaters.

- No native mobile app — a responsive web app only.
- No barcode or RFID hardware; bins carry typed short IDs.
- No public-facing catalog.

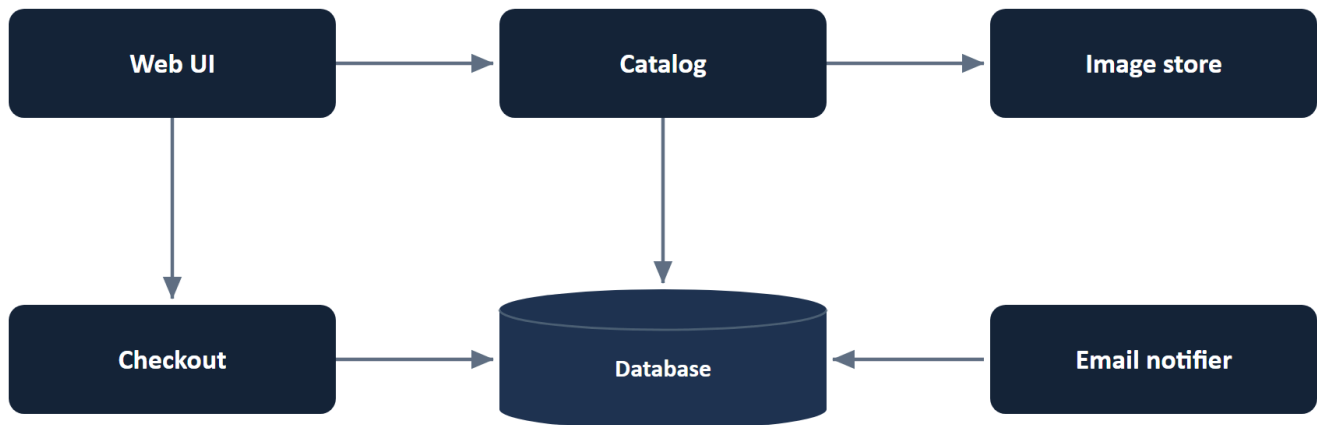
3. System-context diagram



4. Architecture and components

COMPONENT	RESPONSIBILITY	TALKS TO	INTERFACE (ROUGH)
Web UI	Server-rendered pages for search, item detail, add item, check-out. Downscales photos in the browser before upload.	Catalog, Checkout	HTML forms; HTMX partial updates
Catalog	Owns items, categories, bins, and photos. Search by name, category, and size.	Database, Image store	Django views and models
Checkout	Handles check-outs.	Database	TBD
Image store	Saves and serves photos.	Local disk	A storage interface, so object storage can replace the disk later
Auth	Two roles: wardrobe lead and volunteer.	Database	Django’s built-in authentication

Backstage — components



Core flow, traced end to end: a volunteer checks out a costume

1. The volunteer searches “bustle” in the Web UI.
2. Catalog queries the database and returns matching items with photos and bin locations.
3. The volunteer opens an item and taps **Check out**, choosing the actor and show from lists and a due date.
4. Checkout writes a checkout record and marks the item as out.
5. The Web UI shows the item as out, with the actor’s name and due date.

The failure case

If a request fails, the user sees an error message and can try again.

5. Data model

ENTITY	KEY FIELDS	RELATIONSHIPS	MVP OR ROADMAP
Item	name, category, size, condition, bin, photo	belongs to one Bin; has many Checkouts	MVP
Bin	short ID, description	has many Items	MVP
Show	title, opening date	has many Checkouts	MVP
Person	name, phone	has many Checkouts	MVP
Checkout	out date, due date, returned date	belongs to one Item, one Person, one Show	MVP
PullList	show, items	belongs to one Show	Roadmap

The Item table in full:

COLUMN	TYPE	NULL	DEFAULT	NOTES
id	bigint	no	auto	primary key
name	varchar(120)	no	—	indexed
category	varchar(40)	no	'costume'	choices in <code>constants.py</code>
size_label	varchar(20)	yes	NULL	free text
chest_in	numeric(5,2)	yes	NULL	
waist_in	numeric(5,2)	yes	NULL	
inseam_in	numeric(5,2)	yes	NULL	
condition	smallint	no	3	1–5
bin_id	bigint	no	—	FK → bin.id, ON DELETE PROTECT
photo_path	varchar(255)	yes	NULL	relative to MEDIA_ROOT
created_at	timestampz	no	now()	
updated_at	timestampz	no	now()	auto-updated

Search, as implemented:

```
def search(request):
    q = request.GET.get("q", "").strip()
    items = Item.objects.all()
    if q:
        items = items.filter(Q(name__icontains=q) | Q(category__icontains=q))
    if request.GET.get("size"):
        items = items.filter(size_label__iexact=request.GET["size"])
    items = items.select_related("bin").order_by("name")[:50]
    return render(request, "catalog/results.html", {"items": items})
```

6. Technology choices and alternatives considered

Decision: PostgreSQL for the database

- **Chosen:** PostgreSQL
- **Alternative considered:** SQLite, which would remove a service from the VPS entirely
- **Why the chosen option wins, given our goals:** check-outs at strike night happen concurrently — the proposal's observation had three volunteers returning items at once — and SQLite serializes writes. Postgres also matches the Django tooling both authors already know.
- **Evidence:** the strike-night observation; neither author has run SQLite under concurrent writes, so this is a reasoned choice rather than a measured one.
- **What would change our mind:** if the VPS's memory can't hold Postgres alongside the app, SQLite with write retries is the fallback.

Decision: downscale photos in the browser before upload

- **Chosen:** client-side downscaling to 1600 px
- **Alternative considered:** upload full resolution and resize on the server
- **Why the chosen option wins, given our goals:** the shop's wifi is the bottleneck, not the server. Resizing on the server still sends the full image over the weak connection.
- **Evidence:** the CP3 spike — full-resolution upload had a 71 s median with 3 of 10 failures on a throttled connection; downscaled upload had a 48 s median with none.
- **What would change our mind:** nothing short of the theater getting better wifi.

Other choices

Django 5.1.2, django-htmx 1.19.0, Pillow 10.4.0, gunicorn 23.0.0, PostgreSQL 16.4, Ubuntu 24.04 LTS on a \$6/month VPS.

7. Risks and open questions

RISK OR QUESTION	WHY IT MATTERS	OWNER	HOW WE'LL FIND OUT, AND BY WHEN
HEIC photos from iPhones don't render	Half the volunteers use iPhones; the core feature silently breaks	J. Okafor	Test on a borrowed iPhone — milestone 2
HTMX is new to both of us	Could cost days	A. Reyes	Four-hour learning box; fall back to plain forms
Volunteers keep using the paper notebook	The catalog goes stale	A. Reyes	Co-design the add-item form with the wardrobe lead before building it

8. Implementation plan

MILESTONE	WHAT WORKS WHEN IT'S DONE	DEPENDS ON
1	All models and migrations; admin configured	—
2	All views and URL routes	1
3	All templates and styling	2
4	Photo upload and downscaling	3
5	Deploy to the VPS; seed 60 items	4

Sign-off

REVIEWER	ROLE	DATE	APPROVED
M. Tanaka	Design-review partner	—	☐